

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I Thamaraiselvi S (192511042) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Thamaraiselvi S

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

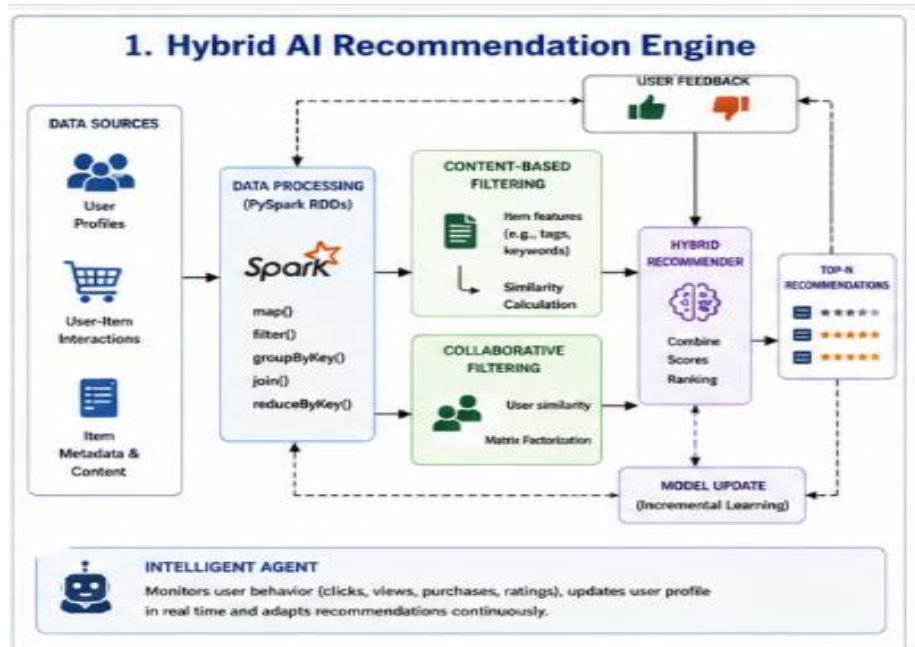
Design Task – Distributed AI Systems Using PySpark RDDs

1. Hybrid AI Recommendation Engine Design

Objective

To design a scalable recommendation system that combines **content-based filtering** and **collaborative filtering** using PySpark RDDs.

Architecture



PySpark RDD Processing

RDDs can distribute large user-item datasets across multiple worker nodes.

Typical operations include:

- `map()` – extract user, item, rating, and feature information.
- `filter()` – remove invalid interactions.
- `groupByKey()` – group interactions by user or item.
- `reduceByKey()` – calculate aggregated ratings or interaction counts.
- `join()` – combine user and item information.

Recommendation Techniques

Content-Based Filtering:

Recommends items similar to those previously viewed or purchased by the user based on item features.

Collaborative Filtering:

Finds users with similar behavior and recommends items they liked.

Hybrid Approach:

Combines both scores to produce a more accurate recommendation.

Intelligent Agents

A recommendation agent observes:

- Clicks
- Searches
- Purchases
- Ratings
- Time spent on items

It updates the user's preference profile and adjusts recommendations based on new feedback.

Conclusion

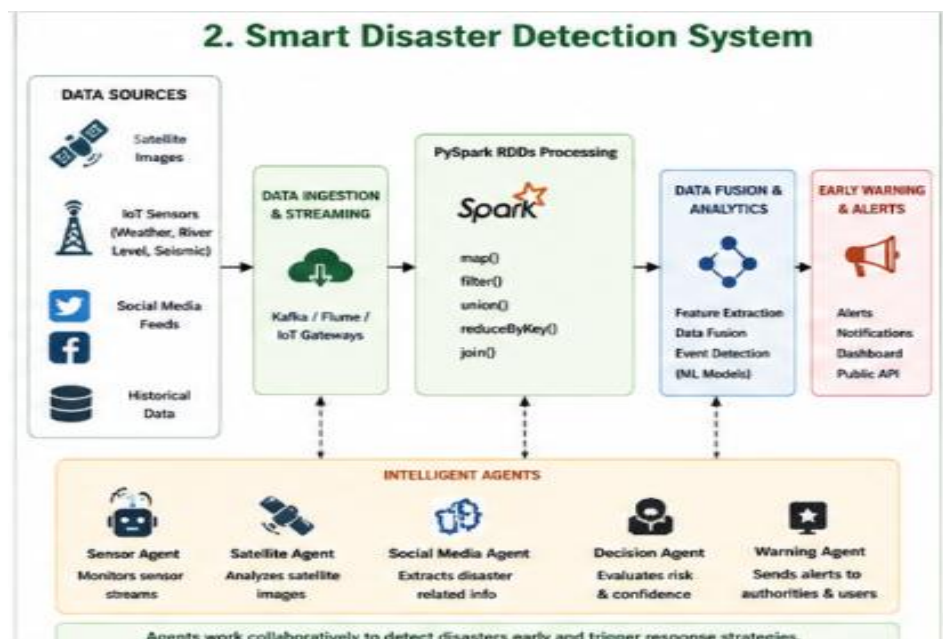
The hybrid approach improves recommendation quality by combining **item similarity** and **user behavior**, while PySpark enables the system to process millions of interactions in parallel.

2. Smart Disaster Detection System

Objective

To develop a distributed AI system that detects floods, earthquakes, and other disasters using **satellite data, sensors, and social-media information**.

Architecture



Data Fusion

Different sources are combined to improve detection reliability.

For example:

- Sensor data detects rising water levels.
- Satellite images identify flooded areas.
- Social-media posts provide information about affected locations.

The system compares these sources and generates a **disaster confidence score**.

PySpark RDD Operations

- `map()` – extract useful information from each data source.
- `filter()` – remove invalid or irrelevant records.
- `union()` – combine datasets from multiple sources.
- `reduceByKey()` – aggregate information by location.
- `join()` – combine sensor and geographical information.

Intelligent Agents

Different agents can perform specialized tasks:

- **Sensor Agent:** Monitors sensor readings.
- **Satellite Agent:** Analyzes satellite information.
- **Social Media Agent:** Detects disaster-related information.
- **Decision Agent:** Combines evidence and evaluates risk.
- **Warning Agent:** Sends alerts to authorities and affected users.

Conclusion

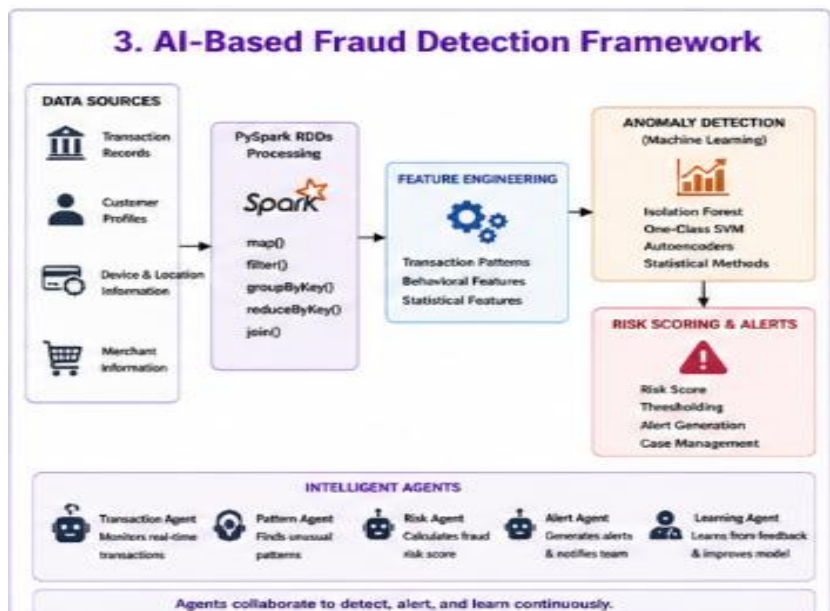
Distributed processing allows huge amounts of disaster-related data to be analyzed quickly. Multi-source data fusion improves detection accuracy, while intelligent agents coordinate detection and warning activities.

3. AI-Based Fraud Detection Framework

Objective

To design a scalable system that detects suspicious financial transactions using **PySpark RDDs and anomaly detection**.

Architecture



Distributed Processing

RDDs divide transactions across multiple worker nodes.

Operations:

- `map()` – extract transaction features.
- `filter()` – remove invalid transactions.
- `groupByKey()` – group transactions by customer/account.
- `reduceByKey()` – calculate transaction frequency or amount.
- `join()` – combine transaction and customer information.

Anomaly Detection

The system can identify unusual:

- Transaction amounts.
- Transaction frequency.
- Locations.
- Device usage.
- Time patterns.

An anomaly score can be calculated for every transaction. Transactions with high scores are sent for further investigation.

Intelligent Agents

- **Transaction Agent:** Monitors incoming transactions.
- **Pattern Agent:** Searches for unusual behavior.

- **Risk Agent:** Calculates fraud risk.
- **Alert Agent:** Generates alerts.
- **Learning Agent:** Uses confirmed fraud/non-fraud feedback to improve future detection.

Conclusion

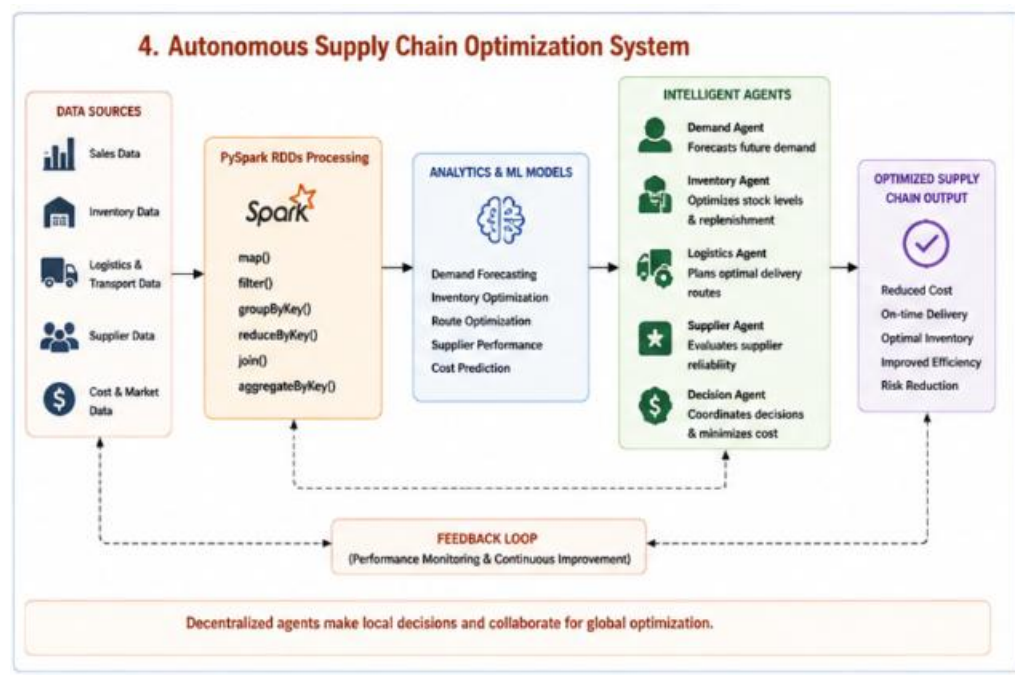
PySpark enables high-volume transaction processing, while distributed anomaly detection identifies suspicious behavior. Intelligent agents provide continuous monitoring and feedback-based improvement.

4. Autonomous Supply Chain Optimization System

Objective

To design a distributed AI system for **inventory management, logistics optimization, and demand forecasting**.

Architecture



RDD Transformations

- `map()` – convert raw records into useful key-value pairs.
- `filter()` – remove incorrect or unnecessary records.
- `groupByKey()` – group sales by product/location.
- `reduceByKey()` – calculate total sales and inventory.
- `join()` – combine inventory, sales, and transportation data.

Intelligent Agents

Demand Agent: Forecasts future product demand.

Inventory Agent: Determines when and how much stock should be replenished.

Logistics Agent: Selects efficient transportation routes.

Supplier Agent: Monitors supplier performance and availability.

Decision Agent: Coordinates the agents and selects actions based on cost, demand, and delivery requirements.

Optimization Goals

- Reduce inventory costs.
- Minimize transportation costs.
- Avoid stock-outs.
- Improve delivery time.
- Balance supply and demand.

Conclusion

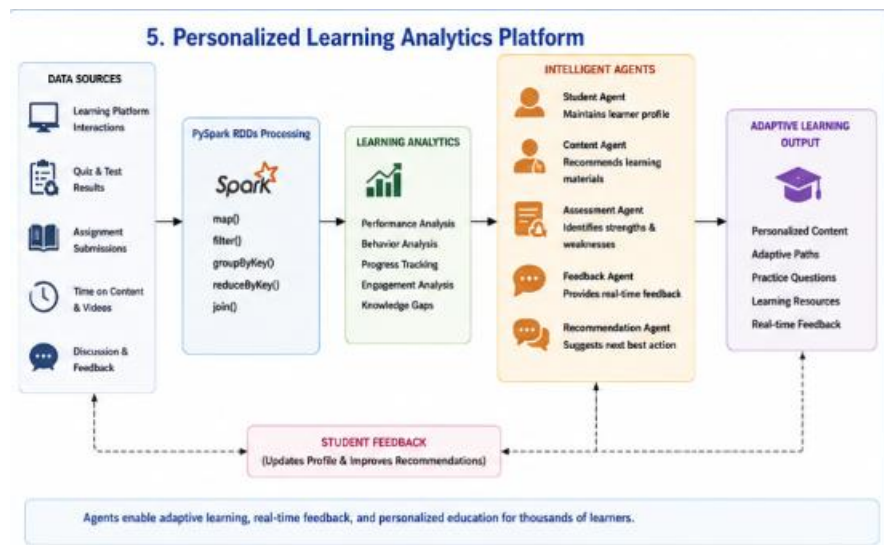
PySpark provides scalable processing of supply-chain data, while multiple intelligent agents make decentralized decisions and coordinate with one another to improve overall efficiency.

5. Personalized Learning Analytics Platform

Objective

To design an AI-based educational platform that analyzes learning behavior and provides **personalized content and real-time feedback** to thousands of students.

Architecture



RDD Processing

RDDs can process learning records distributed across many nodes.

Examples:

- `map()` – extract student performance features.
- `filter()` – remove invalid records.
- `groupByKey()` – group activities by student.
- `reduceByKey()` – calculate average scores.
- `join()` – combine performance with course/content information.

Intelligent Agents

Student Agent: Maintains an individual learning profile.

Content Agent: Selects suitable learning materials.

Assessment Agent: Identifies strengths and areas needing improvement.

Feedback Agent: Provides immediate feedback after quizzes and activities.

Recommendation Agent: Recommends the next lesson or practice activity.

Adaptive Learning

For example:

Low Quiz Score



Agent detects weak topic



Recommends easier explanation



Provides practice questions



Student attempts quiz



Agent updates learning profile

The system continuously adapts content based on the learner's **performance, progress, and interaction history**.

Conclusion

PySpark enables scalable analysis of learning data from thousands of students. Intelligent agents continuously monitor learning behavior and personalize content, recommendations, and feedback for each learner.